

Basic Python Handbook for QA/SDET Interviews (2026 Edition)

Single Volume - Beginner-Friendly Explanations Plus Real-Time Practical Interview Q&A and Programs, Covering Core Python and Basic OOP

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Handbook	3
SECTION 1 — How Python Works & The Basics	4
1.1 Python Is Interpreted, Not Compiled Like Java	4
1.2 Dynamic Typing — Variables Don't Have a Fixed Type	4
Real-Time Interview Q&A	5
SECTION 2 — Variables & Data Types	5
2.1 The Core Built-In Types: int, float, str, bool, None	5
2.2 Type Checking and Conversion	6
Real-Time Interview Q&A	7
SECTION 3 — Operators & Control Flow	7
3.1 if/elif/else and Python's Indentation Rule	7
3.2 for and while Loops, and Python's range()	8
Real-Time Interview Q&A	9
SECTION 4 — Strings	9
4.1 Strings Are Immutable, and Indexing/Slicing	9
4.2 Common String Methods You'll Use Constantly	10
Real-Time Interview Q&A	11
SECTION 5 — Lists & Tuples	11
5.1 Lists — Python's Everyday Resizable Collection	11
5.2 List Comprehensions — a Compact Way to Build a New List	12
5.3 Tuples — an Immutable List	13
Real-Time Interview Q&A	13
SECTION 6 — Dictionaries & Sets	14
6.1 Dictionaries — Key-Value Pairs	14
6.2 Sets — Unique Values, No Duplicates	15
Real-Time Interview Q&A	15
SECTION 7 — Functions	16
7.1 Defining Functions, Parameters, and Return Values	16
7.2 *args and **kwargs — Accepting a Flexible Number of Arguments	16

Real-Time Interview Q&A	17
SECTION 8 — Basic OOP Part 1: Classes, Objects & self	18
8.1 Classes and Objects — the Same Idea as Java, Different Syntax	18
8.2 <code>__init__</code> — Python's Constructor	18
Real-Time Interview Q&A	19
SECTION 9 — Basic OOP Part 2: Inheritance & Polymorphism	20
9.1 Inheritance — <code>class Child(Parent):</code>	20
9.2 Polymorphism — Method Overriding, Python-Style	20
Real-Time Interview Q&A	21
SECTION 10 — Basic OOP Part 3: Encapsulation	22
10.1 Python's Naming-Convention Approach to Private Fields	22
10.2 Properties — Controlled Access Without Java-Style Getters/Setters Everywhere .	23
Real-Time Interview Q&A	24
SECTION 11 — Exception Handling	24
11.1 <code>try, except, else, finally</code>	24
11.2 Raising Your Own Exceptions	25
Real-Time Interview Q&A	25
SECTION 12 — Real-Time Programming Questions, Part 1	26
12.1 Reverse a String	26
12.2 Check if a String Is a Palindrome	27
12.3 Sorting — From Scratch, and Using Built-Ins	27
12.4 Searching — Linear and Binary Search	28
12.5 List Comprehension Practice — Common Transform/Filter Questions	29
SECTION 13 — Real-Time Programming Questions, Part 2	30
13.1 Find Duplicate Elements in a List	30
13.2 Check if Two Strings Are Anagrams	30
13.3 Fibonacci Series	31
13.4 FizzBuzz	32
13.5 Find the Most Frequent Word in a Sentence or Log File	32
13.6 Remove Duplicates From a List While Preserving Order	33

How to Use This Handbook

Python is your secondary skill, and you're a beginner - so this handbook deliberately stays at the level interviewers actually use for a secondary-skill check: core syntax, basic data structures, basic OOP, and the same class of practical coding questions (palindrome, reverse string, sorting, searching) you already have in your Core Java handbook, just in Python. It does not go into advanced territory you're unlikely to be asked about as a secondary skill - no decorators, generators, metaclasses, async, or GIL-level internals. If an interviewer does push further than this, it's very likely into "how would you use this for test automation" territory (e.g., pytest, requests) rather than deep language internals - flag it if that happens and this can be extended.

Same format as your other handbooks: each concept gets a plain-English explanation with code, followed by Real-Time Interview Q&A in the same practical, scenario-driven style as your Playwright, Cypress, and Core Java handbooks. The final two sections are almost entirely hands-on programs - the "write it live" questions - covering the ones you specifically asked for (palindrome, sorting, searching, string reverse) plus the other common ones that travel with them.

SECTION 1 — How Python Works & The Basics

1.1 Python Is Interpreted, Not Compiled Like Java

Level: Foundational | **You'll Use This:** Understanding this once removes confusion about how Python “just runs”

Why It Matters You already know from your Core Java handbook that Java compiles to byte-code, then runs on a JVM. Python works differently, and interviewers sometimes ask this exact comparison specifically because you likely know Java — it's an easy way for them to check real understanding versus memorized facts.

Plain-English Explanation Python code is **interpreted** — the Python interpreter reads and executes your .py file line by line (technically, it compiles to an intermediate bytecode internally too, but this happens automatically and invisibly, with no separate javac-style step you ever run yourself). There's no separate “compile” command you run before “run” — you just run the file directly, and errors in code that hasn't been reached yet won't show up until execution actually gets there.

Code Example

```
# hello_framework.py
print("Framework initialized")
```

```
# Run directly – no separate compile step like javac + java
# $ python hello_framework.py
# Output: Framework initialized
```

```
# A key difference from Java: this file has an error on line 2,
# but line 1 still runs and prints before Python discovers the problem
print("This will print first")
print(undefined_variable) # NameError – but only discovered when this line
↪ actually executes
```

Common Mistake Assuming Python checks your entire file for errors before running anything, the way Java's compiler does — it doesn't. A typo or undefined variable deep in a rarely-used function might not surface until that specific code path actually runs, sometimes much later than you'd expect.

Quick Practice Tip Because Python won't catch many mistakes until that exact line runs, it's worth actually exercising every code path (a good argument for tests!) rather than assuming “it ran without errors” means “it's all correct.”

1.2 Dynamic Typing — Variables Don't Have a Fixed Type

Level: Foundational | **You'll Use This:** Constantly

Why It Matters This is the single biggest everyday difference from Java, and it's asked constantly precisely because it changes how you write and debug code.

Plain-English Explanation In Java, you declare a variable's type once (`String name = "chrome";`) and it can never hold anything else. In Python, a variable is just a name pointing to a value — the *value* has a type, not the variable itself, so the same variable name can point to a string, then later point to a number, with no error at all. Python still checks types at runtime (calling a string method on an integer fails), it just doesn't lock a variable to one type upfront the way Java does.

Code Example

```
browser_name = "chrome"    # browser_name currently points to a string
print(type(browser_name))  # <class 'str'>

browser_name = 3           # completely legal – now points to an integer
↪ instead
print(type(browser_name))  # <class 'int'>

retry_count = "3"         # a string, not a number, even though it looks
↪ numeric
total = retry_count + 2    # TypeError – can't add a string and an int directly
```

Common Mistake Assuming a variable holding a number is safe to use in arithmetic without checking — if that value actually came from user input, a file, or an API response, it might be a string that merely *looks* like a number, and Python won't automatically convert it for you.

Quick Practice Tip When a value's origin isn't guaranteed (config files, API responses, `input()`), explicitly convert it with `int(value)` or `str(value)` rather than assuming it's already the type you expect.

Real-Time Interview Q&A

Q1. You come from a Java background. What's the most important practical difference you'd tell someone about switching to writing Python test scripts?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★ (a very common opening question when Python is a secondary skill)

Why Interviewers Ask This This is a low-pressure way to check genuine hands-on Python exposure versus someone who's only read about it — a candidate who's actually written Python code will have a specific, practical answer, not a textbook one.

Practical Answer “The biggest practical adjustment for me was dynamic typing — in Java, I declare a variable's type once and the compiler enforces it everywhere; in Python, a variable is just a label that can point to any type of value, and type mismatches only surface at runtime, not compile time. This means I lean more on actually running and testing code paths, since Python won't catch a type error until that exact line executes — there's no compiler warning me upfront the way there would be in a Java IDE. Indentation also replaces Java's curly braces for defining code blocks, which took a bit of adjustment but becomes natural quickly.”

Follow-Up Questions 1. Does Python have any way to add type hints for better tooling support, even though it's still dynamically typed underneath? 2. What kind of bug have you personally hit (or would expect to hit) specifically because of dynamic typing?

SECTION 2 — Variables & Data Types

2.1 The Core Built-In Types: `int`, `float`, `str`, `bool`, `None`

Level: Foundational | **You'll Use This:** Constantly

Why It Matters These five cover almost every simple value you'll handle in test scripts — knowing them precisely (especially `None`) avoids a lot of early confusion.

Plain-English Explanation `int` — whole numbers. `float` — decimal numbers. `str` — text, always written in quotes (single or double, Python treats them identically). `bool` — `True` or `False` (capitalized, unlike Java's lowercase `true/false`). `None` — Python's specific way of representing "no value at all," similar in spirit to Java's `null`, but it's a real, distinct object, and you check for it with `is None`, not `== None`.

Code Example

```
retry_count = 3          # int
wait_time = 2.5          # float
browser_name = "chrome"  # str
is_headless = True       # bool — capitalized True/False
last_result = None       # None — represents "nothing here yet"

print(type(retry_count)) # <class 'int'>

if last_result is None:  # correct way to check for None
    print("No result yet")
```

Common Mistake Writing `if last_result == None:` instead of `if last_result is None:` — this often works in practice, but `is` is the correct, idiomatic check for `None` specifically, since it compares identity (is this the exact same singleton `None` object) rather than equality, which is both more precise and the convention every Python style guide recommends.

Quick Practice Tip Always use `is None` / `is not None`, never `== None` — this is one of the most consistently enforced style rules in real Python codebases.

2.2 Type Checking and Conversion

Level: Foundational | **You'll Use This:** Frequently

Why It Matters Config values, environment variables, and command-line arguments almost always arrive as strings even when they represent numbers — converting them correctly is a daily practical need.

Plain-English Explanation `type(value)` tells you a value's actual type. `int()`, `float()`, and `str()` convert between types explicitly — `int("42")` gives you the actual number 42, not the string "42". Converting a non-numeric string (`int("abc")`) raises a `ValueError`.

Code Example

```
retry_count_str = "3"          # arrived as a string, e.g. from a config file
retry_count = int(retry_count_str) # now an actual int, usable in arithmetic

print(retry_count + 2)        # 5 — works now that it's a real int

age_input = "twenty-five"
age = int(age_input)          # raises ValueError: invalid literal for int()

# isinstance() is the idiomatic way to check a value's type before using it
def process_value(value):
    if isinstance(value, str):
        print("Got a string:", value)
    elif isinstance(value, int):
        print("Got a number:", value)
```

Common Mistake Using `type(value) == str` instead of `isinstance(value, str)` — `isinstance()` is the preferred, idiomatic approach in real Python code, and it also correctly handles subclasses, which a direct `type()` comparison would miss.

Quick Practice Tip Default to `isinstance()` for type checks in your own code; reserve `type()` mainly for quick debugging/printing, not for actual conditional logic.

Real-Time Interview Q&A

Q1. A test reads a timeout value from a config file and adds it to a calculation, and the script crashes with a `TypeError`. What’s the likely cause, and how do you fix it?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This This is a genuinely common real bug in test automation scripts specifically, and it directly checks whether you understand that config/file/environment values are strings by default until explicitly converted.

Practical Answer “This is almost always because the value read from the config file arrived as a `str`, not an `int` or `float`, even if it visually looks like a number — Python doesn’t automatically convert it just because the content happens to be numeric-looking text. Adding a string to a number directly raises a `TypeError`, since Python won’t silently guess what you meant. The fix is explicitly converting it with `int(timeout_value)` or `float(timeout_value)` right after reading it from the config, ideally with a `try/except ValueError` around that conversion in case the config value is ever malformed.”

Follow-Up Questions 1. What’s the difference between the errors you’d get from `int("abc")` versus `"abc" + 5`? 2. Where in a real framework would you put this kind of defensive conversion — right at config-loading time, or right before each use?

SECTION 3 — Operators & Control Flow

3.1 if/elif/else and Python’s Indentation Rule

Level: Foundational | **You’ll Use This:** Constantly

Why It Matters Python uses indentation itself to define code blocks — no curly braces at all — and getting this wrong is the single most common early syntax error for anyone coming from Java.

Plain-English Explanation Where Java uses `{ }` to mark a block of code belonging to an `if`, Python uses consistent indentation (4 spaces is the near-universal convention) — everything indented under an `if/elif/else` belongs to that branch. `elif` is Python’s word for “else if” (no separate `else if` keywords like Java).

Code Example

```
status_code = 404

if status_code == 200:
    print("Request succeeded")
elif status_code == 404:
    print("Resource not found")
else:
```

```

    print("Unexpected status:", status_code)

# Inconsistent indentation is a real IndentationError, not just a style
    ↪ complaint
if status_code == 200:
    print("OK")
    print("This line is indented wrong") # IndentationError

```

Common Mistake Mixing tabs and spaces for indentation within the same file — this can look identical in some editors but causes a real `TabError` or `IndentationError` in Python, since it cares about indentation being *exactly* consistent, not just visually similar.

Quick Practice Tip Configure your editor to insert spaces (not tabs) when you press Tab, and to show whitespace characters — this eliminates the entire tabs-vs-spaces class of bug before it happens.

3.2 for and while Loops, and Python's range()

Level: Foundational | **You'll Use This:** Constantly

Why It Matters Python's for loop is fundamentally different from Java's index-based for — it's always "for each item in this iterable," and `range()` is how you get old-style counting behavior when you actually need it.

Plain-English Explanation Python's for loop iterates directly over the items in a collection — there's no separate "index-based for loop" syntax like Java's `for (int i = 0; ...)`. To loop a specific number of times, or by index, you loop over `range(n)`, which generates a sequence of numbers. `while` works the same conceptually as in Java — repeats as long as a condition stays True.

Code Example

```

roles = ["admin", "analyst", "auditor"]

for role in roles:           # iterates directly over each item – the default,
    ↪ most common style
    print("Testing as:", role)

for i in range(5):           # 0, 1, 2, 3, 4 – Python's way to do a counted
    ↪ loop
    print("Attempt", i)

for i in range(len(roles)):   # index-based, when you genuinely need the index
    print(i, roles[i])

# while – same concept as Java
attempts = 0
while attempts < 3:
    print("Retry attempt", attempts)
    attempts += 1 # Python has no ++ operator – must write += 1 explicitly

```

Common Mistake Writing `attempts++` out of Java habit — Python has no `++` or `--` increment/decrement operators at all; you must write `attempts += 1` or `attempts = attempts + 1` explicitly.

Quick Practice Tip Default to `for item in collection:` (the direct, “for-each” style) whenever you don’t specifically need the index; reach for `range(len(collection))` only when the index itself matters.

Real-Time Interview Q&A

Q1. You’re converting a Java-style index-based loop to Python, and it works but feels “wrong” stylistically. What would a Python-idiomatic reviewer suggest instead?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This This checks whether you can write Python that reads like Python, not like translated Java — a common signal interviewers look for from developers with a strong background in another language.

Practical Answer “If the loop only reads elements by index and doesn’t actually need the index for anything (like printing a message per role), I’d change it to iterate directly: `for role in roles:` instead of `for i in range(len(roles)): print(roles[i])`. This is both more idiomatic and slightly clearer, since it removes an unnecessary layer of indexing. I’d only keep the index-based form if the logic genuinely needs the position — like comparing an item to the one before or after it, or printing ‘1 of 5’, ‘2 of 5’, and so on.”

Follow-Up Questions 1. If you do need both the item *and* its index, what’s the idiomatic Python way to get both together in one loop (hint: `enumerate()`)? 2. How would you loop over two lists in parallel, item by item, the Python-idiomatic way?

SECTION 4 — Strings

4.1 Strings Are Immutable, and Indexing/Slicing

Level: Foundational | **You’ll Use This:** Constantly

Why It Matters Just like Java (Section 9.1 of your Core Java handbook), Python strings are immutable — but Python’s slicing syntax for extracting parts of a string is genuinely different and extremely commonly used.

Plain-English Explanation A string can’t be changed in place — every “modifying” operation returns a brand-new string. Indexing (`string[i]`) gets a single character; Python also supports negative indices (`string[-1]` is the last character). Slicing (`string[start:end]`) extracts a substring from start up to (but not including) end.

Code Example

```
browser_name = "chrome"

print(browser_name[0])      # 'c' – first character
print(browser_name[-1])     # 'e' – last character, negative indexing
print(browser_name[1:4])    # 'hro' – characters from index 1 up to (not
    ⇨ including) 4
print(browser_name[:3])     # 'chr' – from the start up to index 3
print(browser_name[3:])     # 'ome' – from index 3 to the end

browser_name[0] = "C"      # TypeError – strings don't support item
    ⇨ assignment, they're immutable
```

```
upper_name = browser_name.upper() # returns a NEW string, original is
↳ untouched
print(browser_name) # still "chrome"
print(upper_name) # "CHROME"
```

Common Mistake Trying to change a single character in place (`browser_name[0] = "C"`) — this raises a `TypeError` immediately, since strings don't support item assignment at all; you'd need to build a new string instead (e.g., slicing around the character and concatenating the replacement).

Quick Practice Tip Slicing is one of the most-used Python idioms — practice reading `string[a:b]` as “everything from index `a` up to, but not including, index `b`” until it's automatic.

4.2 Common String Methods You'll Use Constantly

Level: Foundational | **You'll Use This:** Constantly

Why It Matters Test scripts constantly clean, split, and search text — API responses, log lines, config values — and these methods cover the overwhelming majority of that daily work.

Plain-English Explanation `.strip()` removes leading/trailing whitespace. `.split(delimiter)` breaks a string into a list of pieces. `.join(list)` does the reverse — combines a list of strings into one, with a specified separator. `.replace(old, new)` swaps occurrences of a substring. `.lower()/upper()` change case. `in` checks whether a substring exists inside another string.

Code Example

```
log_line = "  ERROR: Element not found  "

cleaned = log_line.strip()           # "ERROR: Element not found"
parts = cleaned.split(":")          # ["ERROR", " Element not found"]

roles = ["admin", "analyst", "auditor"]
joined = ", ".join(roles)           # "admin, analyst, auditor"

if "ERROR" in cleaned:              # substring check — very common in
↳ log parsing
    print("Found an error line")

fixed = cleaned.replace("ERROR", "WARNING") # "WARNING: Element not found"
print(cleaned.lower())              # "error: element not found"
```

Common Mistake Forgetting that `.split()` with no arguments splits on *any* whitespace (spaces, tabs, newlines) and collapses multiple consecutive spaces automatically, while `.split(" ")` splits strictly on single space characters — these can give different results on messy real-world text with irregular spacing.

Quick Practice Tip When parsing log lines or messy real-world text, prefer plain `.split()` (no argument) over `.split(" ")` unless you specifically need the stricter single-space behavior.

Real-Time Interview Q&A

Q1. You need to extract just the error type from a log line like "2026-01-15 ERROR: TimeoutException occurred". Walk through how you'd do it.

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This A very practical, log-parsing-flavored question checking whether you can combine `.split()` and indexing fluently, which is genuinely daily work in automation.

Practical Answer "I'd split the line on ':' first to separate the message from everything before it, then split the first part on whitespace to isolate the word 'ERROR' if needed, or more directly, split the whole line on spaces and index into the pieces I know the position of. For something this specific, I'd actually reach for 'ERROR:' in `log_line` combined with `.split('ERROR:')[1].strip()` to grab everything after the marker cleanly, which is more robust than counting fixed character positions if the date format's length ever changes."

Code Example

```
log_line = "2026-01-15 ERROR: TimeoutException occurred"

if "ERROR:" in log_line:
    error_detail = log_line.split("ERROR:")[1].strip()
    print(error_detail)  # "TimeoutException occurred"
```

Follow-Up Questions 1. How would this need to change if the log line sometimes said WARNING: instead of ERROR:, and you needed to handle both? 2. Would you consider using a regular expression (re module) instead for more complex log formats — when would that be worth the extra complexity?

SECTION 5 — Lists & Tuples

5.1 Lists — Python's Everyday Resizable Collection

Level: Foundational | **You'll Use This:** Constantly

Why It Matters A Python list plays the same role as Java's `ArrayList` (from your Core Java handbook, Section 11.1) — it's the single most-used collection type, holding test data, results, and anything else that's naturally a sequence.

Plain-English Explanation A list is an ordered, resizable collection, written with square brackets, that can hold values of any type (even mixed types, though that's usually avoided in practice). It grows and shrinks freely with `.append()` and `.remove()`, unlike a fixed-size structure.

Code Example

```
role_names = ["admin", "analyst", "auditor"]

role_names.append("guest")  # adds to the end — list grows
role_names.remove("admin")  # removes the first matching value
print(len(role_names))      # len(), not .length or .size() — a plain
    ⇨ built-in function

for role in role_names:
    print("Testing as:", role)
```

```

print(role_names[0])           # indexing works just like strings
print(role_names[-1])          # negative indexing too
print(role_names[1:3])          # slicing works on lists too

# List comprehension – a very common, idiomatic Python pattern
uppercase_roles = [role.upper() for role in role_names]
print(uppercase_roles) # ['ANALYST', 'AUDITOR', 'GUEST']

```

Common Mistake Using `.length` (that’s Java’s array syntax) or `.size()` (Java’s collection syntax) instead of the plain built-in function `len(list_name)` — Python doesn’t attach a length method to lists at all; it’s a standalone function that works on strings, lists, and several other types uniformly.

Quick Practice Tip Whenever you need a “count,” reach for `len(...)` as a function call wrapping the object, not a method on the object — this is different from both of Java’s two length conventions you already know.

5.2 List Comprehensions — a Compact Way to Build a New List

Level: Core | **You’ll Use This:** Frequently, once comfortable

Why It Matters This is Python’s equivalent of Java’s `Stream.map().filter()` chains (Section 12.2 of your Java handbook) — an extremely common, idiomatic one-liner for transforming or filtering a list, and interviewers specifically look for whether you use this or fall back to a manual loop out of Java habit.

Plain-English Explanation `[expression for item in iterable if condition]` builds a new list by evaluating `expression` for every item in the original collection, optionally keeping only items where `condition` is true. It replaces the “create an empty list, loop, append” pattern with one readable line.

Code Example

```

cohort_sizes = [50, 250, 800, 30, 500]

# Manual loop version – works, but more verbose than necessary
large_cohorts = []
for size in cohort_sizes:
    if size > 100:
        large_cohorts.append(size)

# List comprehension – same result, one line
large_cohorts = [size for size in cohort_sizes if size > 100]
print(large_cohorts) # [250, 800, 500]

# Comprehension with a transformation, not just a filter
doubled = [size * 2 for size in cohort_sizes]
print(doubled) # [100, 500, 1600, 60, 1000]

```

Common Mistake Writing an overly complex, multi-condition comprehension that becomes hard to read in one line — if a comprehension needs more than one `if` or a nested loop to stay clear, a regular `for` loop is usually the more maintainable, more Pythonic-in-spirit choice, despite comprehensions’ popularity.

Quick Practice Tip If you can't read your own list comprehension back in plain English in one breath, it's a sign to break it into a regular loop instead — comprehensions should make code *more* readable, not less.

5.3 Tuples — an Immutable List

Level: Core | **You'll Use This:** Occasionally

Why It Matters Tuples show up constantly as return values from functions that give back more than one piece of data at once — a very common, practical Python pattern with no direct one-line Java equivalent.

Plain-English Explanation A tuple looks and behaves like a list — ordered, indexable — but it's immutable, written with parentheses instead of square brackets, and once created, its contents can never change. This makes it a natural fit for fixed, related groups of values, like coordinates, or a function returning both a status and a message together.

Code Example

```
credentials = ("admin", "adminPass123") # tuple – a fixed pair that shouldn't
↪ change
print(credentials[0]) # "admin"

credentials[0] = "newuser" # TypeError – tuples don't support item assignment

# A common real pattern – a function returning multiple values as a tuple
def check_login_status(response):
    success = response.get("status") == 200
    message = response.get("message", "Unknown")
    return success, message # this actually returns a tuple: (success,
↪ message)

success, message = check_login_status(api_response) # "unpacking" the tuple
↪ into two variables
```

Common Mistake Trying to modify a tuple's contents the way you would a list — this is always a `TypeError`; if you find yourself needing to change values after creation, you actually wanted a list, not a tuple.

Quick Practice Tip Reach for a tuple specifically when a fixed, small group of values naturally belongs together and should never change — function returns of multiple values are the single most common real use case you'll write yourself.

Real-Time Interview Q&A

Q1. You have a list of test execution times in milliseconds and need only the ones that took longer than 2 seconds, uppercased status labels aside. How would you write this the Python way, not the “translated Java” way?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Directly checks list comprehension fluency and whether a Java-background candidate has actually adjusted to writing idiomatic Python rather than direct translations.

Practical Answer “I’d reach for a list comprehension rather than manually creating an empty list and appending in a loop: `slow_tests = [time for time in execution_times if time > 2000]`. This is the idiomatic Python way to filter a list in one readable line, and it’s functionally identical to a Java Stream’s `.filter()`, just Python’s own syntax for the same underlying idea.”

Code Example

```
execution_times = [1200, 2500, 800, 3100, 1900]
slow_tests = [time for time in execution_times if time > 2000]
print(slow_tests) # [2500, 3100]
```

Follow-Up Questions 1. How would you get both the value *and* its original index for each slow test, in the same comprehension? 2. When would you deliberately choose a regular for loop over a comprehension, even for a simple filter like this?

SECTION 6 — Dictionaries & Sets

6.1 Dictionaries — Key-Value Pairs

Level: Foundational | **You’ll Use This:** Constantly

Why It Matters A Python dict plays exactly the same role as Java’s HashMap (Section 11.3 of your Java handbook) — config lookups, grouping data, storing role-to-credential pairs — and it’s one of the most frequently used structures in real automation scripts.

Plain-English Explanation A dict stores key-value pairs, written with curly braces (`{key: value}`), where each key is unique — assigning to an existing key overwrites its value, it doesn’t create a duplicate entry. Access a value with `dict_name[key]` (raises `KeyError` if missing) or `dict_name.get(key, default)` (returns `default` instead of erroring if missing).

Code Example

```
role_credentials = {
    "admin": "adminPass123",
    "analyst": "analystPass456",
}

print(role_credentials["admin"])           # "adminPass123"
print(role_credentials.get("superuser"))   # None — no error, just None
print(role_credentials.get("superuser", "no-password-set")) #
↳ "no-password-set" — safer, explicit fallback

role_credentials["auditor"] = "auditorPass789" # adds a new key
role_credentials["admin"] = "newAdminPass"     # overwrites the existing
↳ value, no duplicate

for role, password in role_credentials.items(): # .items() gives key AND
↳ value together
    print(role, "->", password)

print(list(role_credentials.keys())) # ['admin', 'analyst', 'auditor']
print(list(role_credentials.values())) # ['newAdminPass', 'analystPass456',
↳ 'auditorPass789']
```

Common Mistake Using `dict_name[key]` directly on a key that might not exist — this raises `KeyError` and crashes the script if the key is genuinely absent, unlike Java’s `Map.get()` which quietly returns `null`. Use `.get(key, default)` whenever a missing key is a realistic possibility.

Quick Practice Tip Default to `.get(key, default)` over direct bracket access `[key]` whenever you’re not 100% certain the key exists — this is the same defensive habit from the Java handbook’s `Map.getOrDefault()`, just Python’s specific syntax for it.

6.2 Sets — Unique Values, No Duplicates

Level: Core | **You’ll Use This:** Occasionally, exactly for deduplication

Why It Matters Same practical use as Java’s `HashSet` and TypeScript’s `Set` from your other handbooks — checking for duplicates, or getting only the distinct values from a list, is essentially free with a set.

Plain-English Explanation A set, written with curly braces but no key-value pairs (just values), automatically enforces uniqueness — adding a value that’s already present has no effect. Converting an existing list to a set (`set(my_list)`) is the fastest way to deduplicate it.

Code Example

```
cohort_statuses = ["active", "active", "archived", "active", "pending"]

unique_statuses = set(cohort_statuses)
print(len(unique_statuses)) # 3 - "active", "archived", "pending"

# A real test assertion: confirm no duplicate cohort IDs exist
row_ids = ["c1", "c2", "c3", "c1"] # c1 appears twice - a real bug in this data
unique_ids = set(row_ids)
assert len(unique_ids) == len(row_ids), "Duplicate cohort IDs found!"
```

Common Mistake Expecting a set to preserve the order items were added in — like Java’s `HashSet`, Python’s set makes no ordering guarantee at all; if order matters, keep using a list and handle deduplication differently (e.g., with `dict.fromkeys()`, which does preserve order).

Quick Practice Tip The moment you need “only the unique values” or “check for duplicates,” reach for `set(existing_list)` and compare its length to the original — the exact same one-line pattern from both your Java and TypeScript handbooks.

Real-Time Interview Q&A

Q1. You’re storing browser names tested this run in a list, but a teammate suggests using a set instead. When would that actually matter?

Difficulty: Beginner-Friendly | **Frequency:** ★★★

Why Interviewers Ask This Checks whether you understand the practical trade-off between list and set (order + duplicates allowed, vs. uniqueness with no guaranteed order) rather than treating them as interchangeable.

Practical Answer “If the requirement is genuinely ‘track which browsers have been tested, with no duplicates mattering,’ a set is the better fit — adding ‘chrome’ twice has no effect, and checking membership (`'chrome' in tested_browsers`) is fast and expresses the intent clearly. If the order tests ran in actually matters, or if duplicates are meaningful data (like

counting how many times each browser was tested), I'd keep it as a list instead, since a set would silently collapse that information."

Follow-Up Questions 1. How would you count how many times each browser was tested, if that count matters, given a set can't do that? 2. What's the time complexity difference between checking `x in my_list` versus `x in my_set` for a large collection, and why?

SECTION 7 — Functions

7.1 Defining Functions, Parameters, and Return Values

Level: Foundational | **You'll Use This:** Constantly

Why It Matters Every reusable piece of test logic — a login helper, a wait utility — is a function, and Python's syntax (`def`, no explicit return type declared) is simpler than Java's but has its own specific rules worth knowing precisely.

Plain-English Explanation A function is defined with `def name(parameters):`, followed by an indented body. Unlike Java, you don't declare a return type — Python figures it out at runtime, and a function with no explicit return statement returns `None` automatically. Parameters can have default values, making them optional when calling.

Code Example

```
def build_dashboard_url(cohort_id, tab="summary"): # 'tab' has a default
    ↪ value — optional to pass
    return f"/dashboards/cohort/{cohort_id}?tab={tab}"

print(build_dashboard_url("123"))                # uses default:
    ↪ "/dashboards/cohort/123?tab=summary"
print(build_dashboard_url("123", "filters"))      # overrides default:
    ↪ "...?tab=filters"

def log_message(message):
    print("LOG:", message)
    # no explicit return statement

result = log_message("Test started")
print(result) # None — functions with no 'return' automatically return None
```

Common Mistake Putting a parameter with a default value *before* one without a default — Python requires all non-default parameters to come first, exactly the same rule as Java's constructor/method parameter ordering from your Core Java handbook (Section 3.2).

Quick Practice Tip Read a function with no return statement as implicitly ending in `return None` — this explains why printing the result of a function that “just prints something” gives you `None`, not the printed text itself.

7.2 `*args` and `**kwargs` — Accepting a Flexible Number of Arguments

Level: Core | **You'll Use This:** Occasionally, and it shows up constantly in library code you'll read

Why It Matters You'll see `*args/**kwargs` in a huge amount of real Python code (including popular testing libraries like `pytest`), so recognizing what they mean — even if you rarely write them yourself as a beginner — is genuinely useful.

Plain-English Explanation `*args` collects any number of extra *positional* arguments into a tuple. `**kwargs` collects any number of extra *keyword* (named) arguments into a dictionary. Both let a function accept a flexible, not-fixed-in-advance number of inputs.

Code Example

```
def log_all(*args):
    for arg in args:
        print("Logged:", arg)

log_all("test started", "step 1", "step 2") # any number of positional
↳ arguments accepted

def create_test_user(**kwargs):
    for key, value in kwargs.items():
        print(key, "=", value)

create_test_user(username="admin", role="analyst", active=True) # any number
↳ of named arguments
```

Common Mistake Confusing `*args` (positional, becomes a tuple) with `**kwargs` (named/keyword, becomes a dictionary) — the single and double asterisks are genuinely different mechanisms, not stylistic variants of the same thing.

Quick Practice Tip As a beginner, you'll read these far more often than you write them — when you see `*args`, `**kwargs` in a function signature, read it as “this function accepts anything, in any combination, and figures out what to do with it internally.”

Real-Time Interview Q&A

Q1. You're writing a `wait_for_element` helper that should work with either a default timeout or a custom one the caller specifies. How would you design the function signature?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This A direct, practical check on default parameter usage — one of the most common real function-design decisions in automation helper code.

Practical Answer “I'd give the timeout parameter a sensible default value directly in the function signature, so most callers don't need to think about it at all, but it's still fully overridable for the specific tests that need a longer or shorter wait.”

Code Example

```
def wait_for_element(locator, timeout=10):
    print(f"Waiting up to {timeout} seconds for {locator}")
    # actual wait logic here

wait_for_element("#login-button") # uses the default: 10 seconds
wait_for_element("#slow-export-btn", timeout=30) # explicitly overridden for
↳ a known-slow element
```

Follow-Up Questions 1. What would happen if you tried to call `wait_for_element(30, "#login-button")` instead — positionally, in the wrong order? 2. How would you make `time-out` a keyword-only argument, forcing callers to always write `timeout=30` explicitly rather than positionally?

SECTION 8 — Basic OOP Part 1: Classes, Objects & self

8.1 Classes and Objects — the Same Idea as Java, Different Syntax

Level: Foundational | **You'll Use This:** Constantly once you're writing Page Objects in Python (e.g., with Selenium)

Why It Matters You already understand classes and objects deeply from Java (Section 5 of your Core Java handbook) — this section is mostly about translating that understanding into Python's specific syntax, not learning the concept from scratch.

Plain-English Explanation A class is still a blueprint; an object is still a specific instance created from it. The biggest syntax differences: Python uses `def __init__(self, ...)`: instead of a same-named constructor method, and every method must explicitly take `self` as its first parameter — Python's explicit version of what Java's `this` does implicitly.

Code Example

```
class LoginPage:
    def __init__(self, driver, url):
        self.driver = driver    # 'self.driver' is the field; 'driver' is the
                               ↪ parameter
        self.url = url

    def navigate_to(self):
        print("Navigating to:", self.url)

# Creating objects — no 'new' keyword in Python, unlike Java
admin_login_page = LoginPage(driver, "https://example.com/login")
admin_login_page.navigate_to()
```

Common Mistake Forgetting `self` as the first parameter in a method definition — every instance method must declare it explicitly (Python doesn't do this invisibly the way Java's `this` works), and forgetting it causes a confusing `TypeError` about too many arguments when the method is called.

Quick Practice Tip Read `self` exactly like Java's `this` — “the specific object this method is running on” — the concept is identical; only the requirement to write it explicitly as a parameter is new.

8.2 `__init__` — Python's Constructor

Level: Foundational | **You'll Use This:** Constantly

Why It Matters `__init__` is Python's equivalent of a Java constructor — it runs automatically when you create an object, and it's where you set up the object's initial state.

Plain-English Explanation `__init__` is a special method (the double underscores mark it as a Python “magic method” with a specific, built-in purpose) that Python calls automatically right after creating a new object. Its job is identical to a Java constructor’s — receive initial values and store them on the object via `self`.

Code Example

```
class Cohort:
    def __init__(self, name, size=100): # default value, just like a regular
        ↪ function parameter
        self.name = name
        self.size = size

    def print_summary(self):
        print(f"{self.name} has {self.size} members")

cohort_a = Cohort("Diabetes Cohort") # uses default size = 100
cohort_b = Cohort("Small Cohort", size=25) # explicit size
cohort_a.print_summary() # "Diabetes Cohort has 100 members"
```

Common Mistake Trying to name the constructor the same as the class, Java-style (`def Cohort(self):`) — Python’s constructor is always specifically named `__init__`, never the class’s own name.

Quick Practice Tip `__init__` parameters work exactly like regular function parameters — they can have default values, following the same “defaults must come after non-defaults” rule from Section 7.1.

Real-Time Interview Q&A

Q1. Coming from Java, you write a Python class and forget `self` as a method parameter. What error do you get, and how do you explain what’s actually happening?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★ (a very common early mistake, and interviewers know it)

Why Interviewers Ask This This is one of the most predictable early mistakes for a Java-background developer, and interviewers ask it specifically to check whether you understand *why* Python requires `self` explicitly, not just that you eventually memorized to include it.

Practical Answer “If I write a method without `self` as the first parameter, calling it on an object gives a `TypeError` saying the method takes some number of arguments but a different number were given — because Python automatically passes the object itself as the first argument to any method called on it, and without `self` in the signature to receive that, there’s a mismatch. Unlike Java, where this is available implicitly inside every instance method with no declaration needed, Python makes this explicit — every instance method must declare `self` as its first parameter, precisely because Python’s design philosophy favors being explicit over implicit wherever reasonably possible.”

Follow-Up Questions 1. What would happen if you called the method directly on the *class* instead of an instance, passing an object manually as the first argument yourself? 2. Is `self` a reserved keyword in Python, or just a very strong convention? What would happen if you named it something else?

SECTION 9 — Basic OOP Part 2: Inheritance & Polymorphism

9.1 Inheritance — `class Child(Parent):`

Level: Core | **You'll Use This:** Frequently, in Page Object base classes

Why It Matters Same practical use as your Core Java handbook's BasePage example (Section 6.1) — a shared base class with common logic, extended by specific pages.

Plain-English Explanation A subclass gains everything from its parent by writing the parent's name in parentheses after the class name: `class LoginPage(BasePage):`. To call the parent's `__init__` (or any parent method) explicitly, you use `super()`, which is Python's version of Java's `super`.

Code Example

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver

    def wait_for_page_load(self):
        print("Waiting for page to load...")

class LoginPage(BasePage): # LoginPage inherits everything from BasePage
    def __init__(self, driver):
        super().__init__(driver) # calls BasePage's __init__ — Python's
        ↪ version of Java's super(...)

    def login(self, username, password):
        self.wait_for_page_load() # inherited method, used directly
        print("Logging in as:", username)

login_page = LoginPage(driver)
login_page.login("admin", "pass123")
```

Common Mistake Forgetting to call `super().__init__(...)` in the subclass's `__init__` — unlike Java, Python doesn't force you to call the parent constructor (there's no compile error for skipping it), so it's easy to accidentally end up with a subclass object missing fields the parent's `__init__` was supposed to set up.

Quick Practice Tip As a habit, always call `super().__init__(...)` as the first line of a subclass's `__init__`, even though Python won't force you to — skipping it silently, unlike Java, is exactly what makes this a sneaky bug rather than an obvious compile error.

9.2 Polymorphism — Method Overriding, Python-Style

Level: Core | **You'll Use This:** Frequently

Why It Matters The same concept from your Core Java handbook (Section 6.2/6.3) — a subclass replacing a parent's method — works in Python too, just with less ceremony (no `@Override` annotation required, though Python does have an optional equivalent).

Plain-English Explanation If a subclass defines a method with the exact same name as one in its parent, the subclass's version replaces the parent's for objects of that subclass — calling

it runs whichever version matches the object's actual type, resolved at runtime, exactly like Java's overriding behavior.

Code Example

```
class BasePage:
    def wait_for_page_load(self):
        print("Generic wait logic")

class DashboardPage(BasePage):
    def wait_for_page_load(self): # overrides the parent's version — same
        ↪ method name
        print("Waiting for dashboard-specific loading spinner")

page = DashboardPage()
page.wait_for_page_load() # "Waiting for dashboard-specific loading spinner"
    ↪ — the override runs

# Even through a variable "typed" conceptually as BasePage (Python doesn't
    ↪ enforce this at the variable level,
# but the same runtime-resolution principle from Java's polymorphism still
    ↪ applies)
def run_page_wait(page):
    page.wait_for_page_load() # runs whichever version the ACTUAL object
    ↪ provides

run_page_wait(DashboardPage()) # runs DashboardPage's version
run_page_wait(BasePage())     # runs BasePage's version
```

Common Mistake Assuming Python needs an explicit `@override`-style annotation the way Java uses `@Override` — Python has no required equivalent; simply defining a same-named method in the subclass is enough to override it, with no keyword or decorator required (though some codebases use `typing.override` as an optional, newer convention purely for tooling/clarity).

Quick Practice Tip Because Python doesn't require any explicit override marker, double-check your spelling carefully — a subclass method named slightly differently from the parent's (a typo) won't override anything at all; it'll just silently become an unrelated new method, with no error to warn you.

Real-Time Interview Q&A

Q1. You override a method in a subclass, but calling it on a subclass object still seems to run the parent's version. What are the likely causes?

Difficulty: Core | **Frequency:** ★★★★★

Why Interviewers Ask This This directly targets the “Python won't warn you about a typo'd override” gotcha from 9.2 — a very real, practical debugging scenario.

Practical Answer “The most likely cause is a typo or mismatch in the method name between the parent and subclass — Python has no compile-time check confirming an ‘override’ actually matches an existing parent method, so a subtly misspelled method name just creates an unrelated new method instead of overriding anything, with zero error or warning. I'd double-check the exact method name spelling in both classes character by character. A second possibility:

the object being called on might not actually be an instance of the subclass at all — maybe it was constructed as the parent class directly somewhere unexpected, in which case the parent's version is correctly the only one that exists for that specific object."

Follow-Up Questions 1. How would you use `type(obj)` or `isinstance(obj, Dashboard-Page)` to quickly confirm which actual class an object is, while debugging this? 2. Does Python offer any tooling or type-checker support (like `mypy`) that could have caught this typo before runtime?

SECTION 10 — Basic OOP Part 3: Encapsulation

10.1 Python's Naming-Convention Approach to Private Fields

Level: Core | **You'll Use This:** Occasionally

Why It Matters Python's approach to private/protected (from Java's Section 8.2) is genuinely different — it's convention-based, not enforced by the language — and this is a very commonly asked comparison question specifically because it surprises people coming from Java.

Plain-English Explanation Python has no real private or protected keywords. Instead, it uses naming conventions: a single leading underscore (`_field`) signals "please treat this as internal, don't touch it from outside," but nothing actually *stops* outside code from accessing it — it's a politeness convention, not enforcement. A double leading underscore (`__field`) triggers Python's "name mangling," making it genuinely harder (though still not impossible) to access from outside the class.

Code Example

```
class Cohort:
    def __init__(self, name, member_count):
        self.name = name                # public – no restriction at all
        self._internal_cache = None    # single underscore – "internal,"
        ↪ please don't touch
        self.__member_count = member_count # double underscore –
        ↪ name-mangled, harder to access

    def get_member_count(self):
        return self.__member_count

cohort = Cohort("Diabetes Cohort", 250)
print(cohort.name)                    # fine – public
print(cohort._internal_cache)        # works – Python doesn't actually stop you,
    ↪ just discourages it
print(cohort.__member_count)          # AttributeError – name mangling makes this
    ↪ specific access fail
print(cohort.get_member_count())      # 250 – the intended, correct way to access
    ↪ it
```

Common Mistake Assuming a single underscore (`_field`) provides real protection the way Java's `private` does — it's purely a convention signal to other developers; Python enforces nothing here, and any code can still access `_field` directly if it chooses to.

Quick Practice Tip Treat a single leading underscore as "this is an internal implementation

detail — respect that boundary even though the language won't force you to," which is exactly the spirit of Python's often-repeated phrase: "we're all consenting adults here."

10.2 Properties — Controlled Access Without Java-Style Getters/Setters Everywhere

Level: Core | **You'll Use This:** Occasionally

Why It Matters Java's convention is to write explicit `getX()/setX()` methods for every field that needs validation (Section 8.1 of your Java handbook). Python has a cleaner mechanism — the `@property` decorator — that achieves the same validation goal while still letting callers use plain attribute-style access syntax.

Plain-English Explanation `@property` lets you define a method that's *accessed* like a plain field (no parentheses needed when reading it), while still running actual code behind the scenes — this is how you add validation to an attribute without forcing every caller to write `.get_member_count()` instead of just `.member_count`.

Code Example

```
class Cohort:
    def __init__(self, name, member_count):
        self.name = name
        self._member_count = member_count

    @property
    def member_count(self):          # accessed like a plain attribute:
        ↪ cohort.member_count
        return self._member_count

    @member_count.setter
    def member_count(self, value):  # runs when you WRITE:
        ↪ cohort.member_count = value
        if value < 0:
            raise ValueError("Member count cannot be negative")
        self._member_count = value

cohort = Cohort("Diabetes Cohort", 250)
print(cohort.member_count)        # 250 – reads like a plain attribute, no () needed
cohort.member_count = 300         # runs the setter's validation logic behind the
    ↪ scenes
cohort.member_count = -5          # raises ValueError – the validation actually ran
```

Common Mistake Writing explicit `get_x()/set_x()` methods everywhere out of Java habit, when Python's `@property` gives the same validation benefit while keeping the calling code's syntax simpler and more idiomatic — most experienced Python reviewers will suggest `@property` over manual getter/setter methods for this exact reason.

Quick Practice Tip Reach for `@property` specifically when you need validation or computed logic behind what should still *look* like simple attribute access from the caller's side — for fields with no special logic at all, plain public attributes (no property, no underscore) are the simplest, most Pythonic default.

Real-Time Interview Q&A

Q1. A Java developer new to Python asks why there's no real private keyword, and whether that means Python's OOP is "less safe." How would you respond?

Difficulty: Beginner-Friendly | **Frequency:** ★★★

Why Interviewers Ask This This checks whether you understand the *philosophy* difference, not just the syntax — a common, genuinely insightful comparison question for anyone coming from a stricter language.

Practical Answer "Python's philosophy here is different, not necessarily less safe — it trusts developers to respect naming conventions (a single underscore for 'internal, please don't touch') rather than having the language forcibly block access the way Java's private does. In practice, this works reasonably well within a team that follows the convention, and Python still gives you a real enforcement mechanism when you actually need it — the double-underscore name-mangling, or more commonly, the @property decorator for validated access. I wouldn't call it 'less safe' so much as 'differently safe' — it optimizes for flexibility and trusting the developer, which fits Python's broader design philosophy."

Follow-Up Questions 1. Can you think of a legitimate, real reason someone might deliberately access a single-underscore "internal" attribute from outside its class, despite the convention? 2. How does this convention-based approach affect how you'd review a teammate's PR touching another class's internals?

SECTION 11 — Exception Handling

11.1 try, except, else, finally

Level: Foundational | **You'll Use This:** Constantly

Why It Matters Just as essential in Python automation scripts as in Java (Section 10.1 of your Java handbook) — handling a missing element or failed API call gracefully instead of crashing the whole script.

Plain-English Explanation Code that might fail goes in try. except SpecificError: catches a specific exception type (Python's word for Java's "exception class"). An optional else block runs only if the try block succeeded with no exception at all. finally always runs regardless — used for cleanup that must happen no matter what.

Code Example

```
try:
    element = driver.find_element("id", "login-button")
    element.click()
except NoSuchElementException as e: # 'as e' captures the exception object,
    ↪ just like Java's 'catch (X e)'
    print("Login button not found:", e)
else:
    print("Click succeeded") # only runs if the try block had no exception
finally:
    driver.quit() # always runs – cleanup that must happen regardless of
    ↪ success/failure
```

Common Mistake Writing a bare except: with no exception type specified — this silently catches every possible exception, including ones you never anticipated (like a genuine bug in

your own code), making real problems just as invisible as expected ones. Always catch specific exception types.

Quick Practice Tip Treat a bare `except:` exactly like Java's empty catch (`Exception e`) `{}` from your Core Java handbook (Section 10.3) — it's the same anti-pattern, just Python's specific syntax for accidentally hiding real bugs.

11.2 Raising Your Own Exceptions

Level: Core | **You'll Use This:** Occasionally

Why It Matters Same practical use as Java's custom exceptions (Section 10.3 of your Java handbook) — giving framework-level failures a clearer, more specific error than a generic one.

Plain-English Explanation `raise SomeException("message")` throws an exception explicitly — Python's equivalent of Java's `throw`. You can define your own custom exception types by creating a class that inherits from `Exception` (or a more specific built-in exception).

Code Example

```
class LoginFailedException(Exception):
    """Raised when the login flow fails for a framework-specific reason."""
    pass

def login(driver, username, password):
    try:
        driver.find_element("id", "username").send_keys(username)
        driver.find_element("id", "login-button").click()
    except NoSuchElementException:
        raise LoginFailedException(
            "Could not locate login form – check if the page loaded correctly"
        )

try:
    login(driver, "admin", "pass123")
except LoginFailedException as e:
    print("Framework-specific failure:", e)
```

Common Mistake Defining a custom exception class but forgetting it should inherit from `Exception` (or an appropriate built-in subclass) — without that inheritance, Python won't recognize it as a valid exception type at all, and `raise` will fail.

Quick Practice Tip A custom exception class can be as simple as `class MyError(Exception): pass` — you don't need to write any custom logic inside it unless you specifically want extra behavior; inheriting from `Exception` alone gives you a fully working, distinctly-named exception type.

Real-Time Interview Q&A

Q1. Your script has a bare `except:` block that's been silently swallowing errors for weeks, and a real bug went unnoticed because of it. How do you explain what went wrong, and how do you fix it?

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This This mirrors the exact empty-catch-block question from your Java handbook (Section 10.3) — testing whether the same discipline transfers to Python’s specific syntax for the identical anti-pattern.

Practical Answer “A bare `except:` catches literally every exception, with no distinction between the specific failure I actually expected (like a missing element) and something completely unrelated, like a genuine typo or logic bug elsewhere in the same `try` block. Since it also has no body doing anything with the caught exception — no logging, no re-raise — the failure disappears entirely, and the script just continues as if nothing happened. The fix is catching only the specific exception type I actually expect and know how to handle, and at minimum logging anything caught, so a real, unexpected bug doesn’t go silently unnoticed the way this one did.”

Follow-Up Questions 1. What’s the difference between `except:` (bare) and `except Exception:` (still broad, but slightly more specific) in Python? 2. How would you go back through an existing codebase to find every bare `except:` block already present?

SECTION 12 — Real-Time Programming Questions, Part 1

The same class of “write it live” questions from your Core Java handbook, in Python. Python’s built-ins make several of these shorter than the Java versions — knowing both the manual approach (to show you understand the mechanics) and the built-in shortcut (what you’d actually write in real code) is exactly what interviewers listen for.

12.1 Reverse a String

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same reason as the Java version — fast, low-pressure check of basic fluency, and whether you know Python’s specific shortcuts.

Approach 1 — Slicing (the Python-idiomatic one-liner)

```
def reverse_string(text):  
    return text[::-1] # slice with a step of -1 – walks the string backward  
  
print(reverse_string("selenium")) # "muineles"
```

Approach 2 — Manual loop (shows you understand the mechanics, not just the shortcut)

```
def reverse_string_manual(text):  
    reversed_text = ""  
    for char in text:  
        reversed_text = char + reversed_text # build the result backward  
    return reversed_text
```

Approach 3 — Built-in `reversed()` plus `join()`

```
def reverse_string_builtin(text):  
    return "".join(reversed(text))
```

Complexity: All $O(n)$. The slicing approach is the fastest in practice due to Python’s internal optimizations, and it’s what you’d actually write — but be ready to explain the manual version too, since some interviewers specifically ask you to avoid the slice shortcut.

Common Mistake Only knowing the `[::-1]` slice trick without being able to explain *why* it works — `[::-1]` means “start at the end, go to the beginning, step backward by 1” — if you can’t explain that, an interviewer will reasonably suspect you memorized it without understanding slicing.

Follow-Up Questions 1. Can you explain exactly what each of the three parts of `[::-1]` (start, stop, step) means? 2. How would you reverse a string without using slicing or the `reversed()` built-in, if explicitly asked to avoid both?

12.2 Check if a String Is a Palindrome

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same as the Java version — checks whether you think about case-sensitivity and non-alphanumeric characters without being told to.

The Python-idiomatic approach

```
def is_palindrome(text):
    cleaned = "".join(char.lower() for char in text if char.isalnum()) #
    ↪ strip spaces/punctuation, lowercase
    return cleaned == cleaned[::-1]

print(is_palindrome("A man, a plan, a canal: Panama")) # True
print(is_palindrome("hello")) # False
```

Manual two-pointer version (if asked to avoid slicing)

```
def is_palindrome_manual(text):
    cleaned = "".join(char.lower() for char in text if char.isalnum())
    left, right = 0, len(cleaned) - 1
    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True
```

Complexity: $O(n)$ for both approaches.

Common Mistake Forgetting `.isalnum()` to filter out spaces and punctuation — without it, “A man, a plan...” would incorrectly return `False`, since the raw string (with spaces and commas) isn’t actually a mirror image of itself even though the underlying letters are.

Follow-Up Questions 1. How would you check if a *number* is a palindrome without converting it to a string at all? 2. What does the generator expression `(char.lower() for char in text if char.isalnum())` actually produce, before `"".join()` consumes it?

12.3 Sorting — From Scratch, and Using Built-Ins

Difficulty: Beginner-Friendly | **Frequency:** ★★★

Why Interviewers Ask This Same purpose as Java’s bubble sort question (Section 13.3 of your Java handbook) — checking nested-loop and complexity reasoning, plus whether you know

Python's built-in sort is what you'd actually use.

Bubble Sort — implemented manually

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Python's
    ↪ tuple-unpacking swap — no temp variable needed
    return arr

print(bubble_sort([5, 2, 8, 1, 9])) # [1, 2, 5, 8, 9]
```

What you'd actually use in real code

```
numbers = [5, 2, 8, 1, 9]
numbers.sort() # sorts the list IN PLACE, returns None
sorted_numbers = sorted(numbers) # returns a NEW sorted list, leaves the
    ↪ original untouched

names = ["Charlie", "Alice", "Bob"]
names.sort(key=str.lower) # sort using a custom key — here,
    ↪ case-insensitive
names.sort(reverse=True) # descending order
```

Complexity: Bubble sort is $O(n^2)$, same as Java. Python's built-in `.sort()`/`sorted()` uses Timsort internally — $O(n \log n)$, stable, and highly optimized.

Common Mistake Confusing `.sort()` (mutates the list in place, returns `None`) with `sorted()` (returns a brand-new sorted list, original untouched) — assigning the result of `.sort()` to a variable gives you `None`, a very common early mistake.

Follow-Up Questions 1. Why is `list.sort()` generally faster than `sorted(list)` when you don't need to preserve the original order? 2. How would you sort a list of dictionaries by one specific key's value (e.g., sorting test results by execution time)?

12.4 Searching — Linear and Binary Search

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same as the Java version — checks whether you understand binary search's precondition (sorted data) and its $O(\log n)$ advantage.

Linear Search

```
def linear_search(arr, target):
    for i, value in enumerate(arr): # enumerate() gives index AND value
    ↪ together
        if value == target:
            return i
    return -1
```

Binary Search

```
def binary_search(sorted_arr, target):
    left, right = 0, len(sorted_arr) - 1
    while left <= right:
        mid = (left + right) // 2  # // is integer (floor) division in Python
        if sorted_arr[mid] == target:
            return mid
        elif sorted_arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

What you'd actually use for a simple membership check

```
numbers = [1, 3, 5, 7, 9]
print(7 in numbers)  # True - Python's 'in' does a linear search internally
                     ↪ for a list
```

Complexity: Linear search $O(n)$. Binary search $O(\log n)$, but only valid on sorted data.

Common Mistake Using `//` when you meant `/` (or vice versa) for the midpoint calculation — `/` in Python 3 always produces a float (e.g., `4/2` is `2.0`, not `2`), which would break list indexing; `//` is the integer/floor division you actually need here.

Follow-Up Questions 1. Why does `sorted_arr[mid]` fail (or behave unexpectedly) if `mid` were accidentally a float instead of an int? 2. Python's built-in `in` operator on a list does a linear scan — what data structure would give you near-instant membership checks instead, and why?

12.5 List Comprehension Practice — Common Transform/Filter Questions

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Interviewers often ask a small batch of these back-to-back specifically to see how fluently you reach for comprehensions instead of manual loops, since it's such a defining, idiomatic Python habit.

Get all even numbers from a list

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
evens = [n for n in numbers if n % 2 == 0]
print(evens)  # [2, 4, 6, 8]
```

Square every number in a list

```
squared = [n ** 2 for n in numbers]  # ** is Python's exponent operator
print(squared)  # [1, 4, 9, 16, 25, 36, 49, 64]
```

Get the lengths of every word in a sentence

```
words = "the quick brown fox".split()
lengths = [len(word) for word in words]
print(lengths)  # [3, 5, 5, 3]
```

Common Mistake Nesting too much logic into one comprehension (multiple conditions, a nested loop, and a transformation all at once) — if it takes more than a few seconds to read back in plain English, break it into a regular for loop instead, exactly the same principle from Section 5.2.

Follow-Up Questions 1. How would you write a comprehension that produces a list of tuples, pairing each word with its length? 2. What's the dictionary-comprehension equivalent syntax, and when would you reach for that instead of a list comprehension?

SECTION 13 — Real-Time Programming Questions, Part 2

13.1 Find Duplicate Elements in a List

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same as the Java version — checks whether you reach for a set instead of a slower nested-loop brute-force approach.

Using a set (the expected, efficient answer)

```
def find_duplicates(items):
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

print(find_duplicates([1, 2, 3, 2, 4, 1])) # [1, 2] (order not guaranteed)
```

A very short alternative using collections.Counter

```
from collections import Counter

def find_duplicates_counter(items):
    counts = Counter(items)
    return [item for item, count in counts.items() if count > 1]
```

Complexity: $O(n)$ for both approaches — a significant improvement over an $O(n^2)$ brute-force nested loop.

Common Mistake Using `items.count(item) > 1` inside a loop over `items` — this looks reasonable but is actually $O(n^2)$ overall, since `.count()` itself scans the whole list every single time it's called, silently reintroducing the slow brute-force approach you were trying to avoid.

Follow-Up Questions 1. What does `Counter` actually return, and how is it different from a plain dict? 2. How would you find duplicates while preserving their original order in the result?

13.2 Check if Two Strings Are Anagrams

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same as the Java version — checks character-frequency thinking, and whether you know Python’s shortcuts for it.

Approach 1 — Sort and compare (short, very Python-idiomatic)

```
def is_anagram(a, b):  
    return sorted(a.lower()) == sorted(b.lower())  
  
print(is_anagram("listen", "silent")) # True
```

Approach 2 — Using Counter (more efficient, avoids sorting)

```
from collections import Counter  
  
def is_anagram_counter(a, b):  
    return Counter(a.lower()) == Counter(b.lower())
```

Complexity: Sort-and-compare is $O(n \log n)$. Counter-based comparison is $O(n)$ — no sorting needed, and Counter objects support direct equality comparison out of the box.

Common Mistake Forgetting `.lower()` on both strings — "Listen" and "silent" would incorrectly fail the check due to case mismatch alone, even though they’re genuinely anagrams of each other ignoring case.

Follow-Up Questions 1. Why does comparing two Counter objects directly with `==` correctly check “same characters, same frequencies” in one line? 2. How would you adapt this to ignore spaces too, for checking if two phrases are anagrams of each other?

13.3 Fibonacci Series

Difficulty: Beginner-Friendly | **Frequency:** ★★★

Why Interviewers Ask This Same as the Java version — checks whether you recognize the naive recursive approach’s exponential blowup.

Naive Recursive (correct, but slow for larger inputs)

```
def fibonacci_recursive(n):  
    if n <= 1:  
        return n  
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

Iterative (efficient)

```
def fibonacci_iterative(n):  
    if n <= 1:  
        return n  
    prev, curr = 0, 1  
    for _ in range(2, n + 1): # '_' is a conventional throwaway variable name  
        ↪ - unused on purpose  
        prev, curr = curr, prev + curr # tuple unpacking updates both at once,  
    ↪ no temp variable  
    return curr
```

Complexity: Naive recursive is $O(2^n)$. Iterative is $O(n)$.

Common Mistake Not recognizing `_` as a deliberate Python convention for “this loop variable, I genuinely don’t need” — some beginners think it’s a typo when they see it in someone else’s code.

Follow-Up Questions 1. What is memoization, and how would `functools.lru_cache` (a Python built-in decorator) fix the recursive version’s performance with almost no code change? 2. At roughly what value of `n` does the naive recursive version become painfully slow, and why?

13.4 FizzBuzz

Difficulty: Beginner-Friendly | **Frequency:** ★★★★★

Why Interviewers Ask This Same universal screening question as the Java version — trivial, but a surprising number of people still get the condition order wrong live.

Implementation

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)
```

Common Mistake Same as the Java version — checking `% 3` and `% 5` as separate top-level conditions *before* `% 15`, causing numbers divisible by both to only ever print “Fizz” and never reach “FizzBuzz”.

Follow-Up Questions 1. How would you write this as a one-line list comprehension instead of a loop with `print()` statements? 2. How would you make the divisors and labels configurable instead of hardcoded 3/5/15?

13.5 Find the Most Frequent Word in a Sentence or Log File

Difficulty: Core | **Frequency:** ★★★★★ (genuinely practical for log analysis)

Why Interviewers Ask This Same practical, automation-relevant purpose as the Java version — and Python’s `Counter` makes this dramatically shorter, which is itself often the point of the question when Python is being compared to Java.

Implementation

```
from collections import Counter

def most_frequent_word(text):
    words = text.lower().split()
    counts = Counter(words)
    return counts.most_common(1)[0][0]  # most_common(1) -> [('word', count)]

logs = "TimeoutException element not found TimeoutException retry failed
↳ TimeoutException"
```

```
print(most_frequent_word(logs)) # "timeoutexception"

# most_common(n) is also how you'd get the TOP N most frequent words directly
print(Counter(logs.lower().split()).most_common(2))
# [('timeoutexception', 3), ('element', 1)]
```

Complexity: $O(n)$ to build the counts, $O(k \log k)$ for `most_common()`'s internal sort where k is the number of distinct words — much shorter code than the manual Java Map-based version, same underlying idea.

Common Mistake Splitting on plain spaces (`.split(" ")`) when punctuation is attached to words — same issue as the Java version; `.split()` with no argument (splitting on any whitespace) is usually safer, though neither handles punctuation attached directly to a word (like "failed.") without extra cleanup.

Follow-Up Questions 1. How would you strip punctuation before counting, so "failed." and "failed" count as the same word? 2. What does `Counter.most_common()` return with no argument at all, versus `most_common(1)`?

13.6 Remove Duplicates From a List While Preserving Order

Difficulty: Core | **Frequency:** ★★★★★

Why Interviewers Ask This A very common practical follow-up once you've shown you know `set()` for deduplication — checks whether you know `set()` alone loses order, and what to use instead when order matters.

The problem with just using `set()`

```
items = ["b", "a", "b", "c", "a"]
print(set(items)) # {'a', 'b', 'c'} — deduplicated, but original order is NOT
↳ guaranteed to be preserved
```

Order-preserving deduplication using `dict.fromkeys()`

```
def remove_duplicates_preserve_order(items):
    return list(dict.fromkeys(items)) # dicts preserve insertion order in
↳ modern Python (3.7+)

print(remove_duplicates_preserve_order(["b", "a", "b", "c", "a"])) # ['b',
↳ 'a', 'c']
```

Manual approach (shows you understand why, not just the trick)

```
def remove_duplicates_manual(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

Complexity: $O(n)$ for both approaches.

Common Mistake Reaching straight for `list(set(items))` when order actually matters for the test or report being generated — this works for deduplication but silently scrambles the original order, which may or may not be acceptable depending on what the result is used for.

Follow-Up Questions 1. Why does `dict.fromkeys(items)` happen to preserve order, given dictionaries are conceptually about keys and values, not sequences? 2. Which approach would you pick if `items` contained values that aren't hashable (like lists), and neither `set` nor `dict` keys could be used at all?
